

SEN-107 Fundamental Data Structures
Assessment Activity SEN-107:00050, SEN-107:00060
Creating and Using Graphs
Using Content-Addressable Storage
Last update: 20 January 2025

Instructions

General Instructions

Write a program that solves the problem described below. Your program should read from the standard input and write to the standard output. We have provided example input and output files. When running your program on the example input file, it should produce the same output as in the example output file. While this example is a good starting point, you should test your program thoroughly, with multiple different inputs too. Your program should correctly handle all possible cases specified below. Correctness will be part of your assessment score. Please also follow the coding standards in ***C_CodingStandards.pdf***. Violating the coding standards will lower your assessment score.

Submit your ***C source code*** only. Your source code must be a single plain text file. Do not submit code as a doc file, pdf file, IDE project, or anything else. We must be able to compile and run your code on my computer, without any warnings or errors.

For convenience, we have provided a template for you, which already contains the skeleton code that handles the input-output. You may implement your solution using any data structure you learn from lectures (or beyond). However, you are **not** allowed to use other external libraries, such as C++ Standard Template Library (STL).

Assignment-Specific Instructions

For this assessment, you should submit a single file called ***cmkl_book.c***; as stated above, we have already provided a template for this file. The example input and output files are ***cmkl_book.in*** and ***cmkl_book.out***, respectively.

Problem Statement: Simple Social Network

Write a program called ***cmkl_book.c*** that implements a simple social network. The program should have the following capabilities:

1. Add a new account, with information: first name, last name, and email address. Email address must be unique. If an account with this email already exists, print an error message. Do not create the new account. (Note that, if an email address had been used for an account but that account was deleted already, then adding a new account with this email address should be successful.)
2. Delete an account, based on the email address. Report whether the deletion is successful or not.
3. Make one account be a friend of another. Specify the two people by their email addresses. If one or both of the accounts do not exist, print an error message. Note that in the CMKLBook social network, the friend relationship is not directional. If A is friends with B, B is also friends with A.
4. Unfriend someone. Specify the two by their email addresses (handle errors similar to #3)

5. Print all the friends of one person (specified by their email address, handle errors similar to #3).
6. List suggestions for new friends for a person (specified by their email address, handle errors similar to #3). The program should suggest all people who are friends of that person's friends. However, you should not suggest anyone who is already the focus person's friend.

The **cmkl_book** program must be able to handle an unlimited number of accounts.

To represent the relationships between friends, you **should use a graph data structure**. You may use implementation of a graph from the lab / demo code, or build your own.

To look up accounts by their email addresses, **you should use a hash table**. You may use implementation of a hash table from the lab / demo code, or build your own.

Input Description

The first line of the input contains a number T denoting the number of operations.

Each of the next T lines denotes one of the two events.

1. If the line starts with "New", it indicates a new-account operation. It will be followed by three strings, the person's email address, first name and last name, respectively.
2. If the line starts with "Delete", it indicates a delete-account operation. It will be followed by a single string, the person's email address.
3. If the line starts with "Add" it indicates an add-friend operation. It will be followed by two strings, the two people's email addresses.
4. If the line starts with "Unfriend" it indicates an unfriend operation. It will be followed by two strings, the two people's email addresses.
5. If the line starts with "Friend", it is asking to print a person's friends. It will be followed by a single string, the person's email address.
6. If the line starts "Suggestion", it is asking to print a person's friend suggestions. It will be followed by a single string, the person's email address.

You can assume that each input name and email address have length at most 50.

Output Description

After each operation, the program should print the following in a new line:

1. New-account operation:
 - If the operation is successful, print "Success".
 - Otherwise, if the email address has already been used, print "Error: Duplicate".
2. Delete-account operation:
 - If the operation is successful, print "Success".
 - Otherwise, if the email address is not associated with any account, print "Error: Not Found".
3. Add-friend operation:
 - If the operation is successful, print "Success".
 - If one or both of the email addresses are not associated with any (active) account, print "Error: Account Not Found".
 - Otherwise, if the two email addresses are the same, print "Error: Can't add friend to self".
 - If the two people are already friends, print "Error: Accounts are already friends".
4. Unfriend operation:
 - If the operation is successful, print "Success".
 - If one or both of the email addresses are not associated with any (active) account, print "Error: Account Not Found".

- Otherwise, if the two email addresses are the same, print “Error: Can’t add unfriend self”.
 - If the two people are not friends, print “Error: Can’t unfriend accounts that are not friends”.
5. Friend operation:
- If the email address is not associated with any (active) account, print “Error: Account Not Found”.
 - If the account has no friend, print “No Friend”
 - Otherwise, print the first and last names of every friend of this person. Each person’s first and last names should be separated by a space, whereas different people names should be separated by a comma and a space (i.e. ", ").
 - The friends can be listed in any order.
 - Each friend should appear only once in the list.
6. Suggestion operation:
- If the email address is not associated with any (active) account, print “Error: Account Not Found”.
 - If the account has no friend suggestion, print “No Friend Suggestion”
 - Otherwise, print the first and last names of every person who is a friend of a friend of this person but not this person’s friend. Each person’s first and last names should be separated by a space, whereas different people names should be separated by a comma and a space (i.e. ", ").
 - The suggested friends can be listed in any order.
 - Each suggested friend should appear only once in the list.

Example Input & Output

Input	Output
20	Success
New donald@temp.com Donald Trump	Error: Duplicate
New donald@temp.com Don Trump	Error: Not Found
Delete pasin@cmkl.ac.th	Success
New elon@temp.com Elon Musk	Success
New biden@temp.com Joe Biden	Success
New kamala@temp.com Kamala Harris	Success
New vance@temp.com JD Vance	Error: Account Not Found
Add donald@temp.com pasin@cmkl.ac.th	Error: Can’t add friend to self
Add donald@temp.com donald@temp.com	Success
Add donald@temp.com biden@temp.com	Success
Add donald@temp.com elon@temp.com	Success
Add biden@temp.com kamala@temp.com	Success
Add vance@temp.com biden@temp.com	Error: Account Not Found
Friend pasin@cmkl.ac.th	Joe Biden, Elon Musk
Friend donald@temp.com	Kamala Harris, JD Vance
Suggestion donald@temp.com	Success
Unfriend elon@temp.com donald@temp.com	Success
Delete donald@temp.com	No Friend
Friend elon@temp.com	No Friend Suggestion
Suggestion elon@temp.com	

Note that there are multiple valid outputs for this problem; in particular, each list of friends and each list of suggested friends can be arbitrarily reordered.